

Writing While Not Reading Too Much

2018-04-01 07:17:15

Original: <https://nicknash.me/2018/04/01/writing-while-not-reading-too-much/>

1 Recap

My [last post](#) described the concurrency construction called Left-Right at a very high level. Left-Right allows for concurrent reading and writing of a data structure with wait-free reads that can run in parallel with a writer. You can think of it as a neat alternative to copy-on-write or a better reader-writer lock, but it's also interesting in its own right.

In this post I'll present an implementation of Left-Right in C# and describe how it works.

2 Starvation

The main features of the Left-Right construction are the following:

1. It's generic, works with any data structure, and doesn't affect the time complexity of any operations.
2. Reads and writes can happen concurrently.
3. Reading is always wait free.
4. No allocation is required, so no garbage is generated, and no garbage collector is required.
5. A writer cannot be starved by readers.

The fifth property – that writers cannot be starved by readers means that a write operation will never be prevented from completing by read operations. It is also the trickiest part of Left-Right.

As a result, for this post, I'm going to describe the so-called "No Version" variation of Left-Right. It satisfies the first four properties, but readers can (in very unlikely scenarios) starve a writer.

Once you've got your head around this version of Left-Right, adding in the starvation freedom property is much easier to understand.

3 Reading

We'll begin by describing how to use Left-Right to read a data structure. This will show the first key idea of Left-Right: It maintains *two* versions of the data-structure, allowing reads to proceed concurrently with a writer.

To keep the description of reads simple, we'll assume the existence of a so-called "Read Indicator", that is used to communicate in-progress reads with potential writers. Here's the interface a read-indicator exposes:

```
interface IReadIndicator
{
    // A reader calls this to tell writers that it is
    // reading the data structure
    void Arrive();
    // A reader calls this to tell writers that
    // it is no longer reading the data structure
    void Depart();
    // Returns true if a read in progress (i.e.,
    // any calls to Arrive without a matching Depart call)
    bool IsOccupied { get; }
}
```

We'll ignore the implementation of `IReadIndicator` for now. Implementations can be very simple – a single atomic counter will do (and be unscalable in the presence of many threads), or they can be quite intricate and more scalable.

With our read-indicator in hand, the implementation of a read looks something like this:

```
// An array of the two instances of the data
// structure we are coordinating access to. For this example, a List
List[] _instances;
// An index (either 0 or 1) that tells readers
// which instance they can safely read.
int _readIndex;
// A read indicator, that a reader can use to inform a potential
// writer that a read is in progress.
IReadIndicator _readIndicator;
int Read()
{
    _readIndicator.Arrive();
    var idx = _readIndex;
    var result = read(_instances[idx]);
    _readIndicator.Depart();
    return result;
}
```

This code is hopefully self-explanatory: readers just record their presence during reading.

4 Writing

Writing is a bit trickier than reading, and goes something like this: Write to the instance of the data structure that we're sure there can be no readers reading, wait for readers to finish reading the other instance, and then write to that instance.

A (toy) implementation would look something like this:

```
void Write()
{
```

```

lock(_writersMutex)
{
    var readIndex = _readIndex;
    var nextReadIndex = Toggle(readIndex);
    write(instances[nextReadIndex]);
    _readIndex = nextReadIndex;
    while(_readIndicator.IsOccupied) ;
    write(instances[readIndex]);
}
}

```

The first thing to note about this is that writes are blocking: there can only be a single write in progress at once, hence the `_writersMutex`.

Next we'll turn to understanding exactly why this write implementation can proceed even in the presence of readers

5 Why This Is Correct

The argument for correctness I'm about to give might seem a bit laboured. I think it's worth it though, because when we switch to the starvation-free version of Left-Right we can use the same tools.

So let's start with some definitions.

Let's define the *end-state* of a reader as the value of `_readIndex` it has witnessed at the time it calls `Depart`. So there are two possible reader end-states: 0 and 1. After a reader calls `Depart` it is said to *complete* and is not defined to be in any state.

Let's also divide readers into two categories: *pre-arrival* readers and *post-arrival* readers. Pre-arrival readers have not called `Arrive` on a read-indicator yet. While post-arrival readers have called `Arrive` on a read-indicator.

Now we can note the key property that makes Left-Right correct: If a writer waits until `_readIndicator.IsOccupied` returns `false` while `_readIndex` is 1 then there cannot be any reader with end-state 0. This is because any pre-arrival reader cannot have yet read `_readIndex` because it has not yet called `_readIndicator.Arrive()`, and hence if it does it must see `_readIndex` as 1. On the other hand, the wait on `_readIndicator.IsOccupied` ensures that any reader with end-state 0 will complete. Of course, a symmetrical argument applies if a writer changes `_readIndex` from 1 to 0.

To be completely explicit, mutual exclusion between readers and writers is assured when a write to proceeds on a given `_readIndex` when the only possible reader end-states are the toggle of `_readIndex`. Moreover the preceding paragraph shows this is exactly the protocol obeyed by `Write`, and so this implementation is correct.

6 Next Time

The Left-Right implementation just described is nearly the real-deal. The unfortunate thing is that under intense read pressure, readers can starve the writer. This is because the wait on `_readIndicator.IsOccupied` may wait on readers that begin after the call to `Write` began. I hope to describe how to lift this limitation in my next post.

Lastly, there is a gap in the explanations above: I've omitted all reference to the required memory ordering on the operations (e.g. so called 'volatile' operations in Java and C# or 'atomic' operations in C++), this is also something I hope to give full details of in a future post. In the mean-time, there is a RelSharp example of this Left-Right implementation, that includes memory fencing here: [StarvationLeftRight.cs](#)